

Rajalakshmi Engineering College

Name: Mahathi B

Email: mahathi.b.2024.csd@rajalakshmi.edu.in

Roll no: 241701029

Phone: 9841411040

Branch: REC

Department: CSD

Batch: 2028

Degree: B.E - CSD

Scan to verify results



2024_28_IV_CSD_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 5_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Anjali is working as a developer for the City Basketball Association, which wants to build a system to track and find the top scorer among basketball players.

Each player's record has:

Player ID (integer) Player Name (string) An array of points scored in 5 matches (integers)

The system must calculate:

The total score of each player (sum of all match points). Identify the highest scorer among all players. If two or more players have the same total score, the one with the lower Player ID is considered the top scorer.

Anjali has been asked to implement this system using:

A class with attributes for player details. A constructor to initialize player details. Getter and Setter methods to retrieve and update player details if required. A method to calculate the total score. Objects of the class to represent players.

Finally, display each player's details and announce the Top Scorer.

Input Format

The first line of input contains an integer N (number of players).

For each player:

- The next line contains the Player ID (integer).
- The following line contains the Player Name (string).
- The next line contains 5 integers separated by spaces (points scored in 5 matches).

Output Format

For each player the output prints the following details:

- Player ID: <player_id>
- Player Name: <player_name>
- Total Score: <total_score>

Finally, print "Top Scorer: <player_name> with <total_score> points"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

10 20 30 40 50

Output: Player ID: 1001
Player Name: Ravi Kumar
Total Score: 150
Top Scorer: Ravi Kumar with 150 points

Answer

// You are using Java
import java.util.Scanner;

```
class Player {  
    private int playerId;  
    private String playerName;  
    private int[] points;  
  
    // Constructor  
    public Player(int playerId, String playerName, int[] points) {  
        this.playerId = playerId;  
        this.playerName = playerName;  
        this.points = points;  
    }  
  
    // Getters  
    public int getPlayerId() {  
        return playerId;  
    }  
  
    public String getPlayerName() {  
        return playerName;  
    }  
  
    public int[] getPoints() {  
        return points;  
    }  
  
    // Setters  
    public void setPlayerId(int playerId) {  
        this.playerId = playerId;  
    }  
  
    public void setPlayerName(String playerName) {  
        this.playerName = playerName;  
    }  
}
```

```

    public void setPoints(int[] points) {
        this.points = points;
    }

    // Method to calculate total score
    public int calculateTotalScore() {
        int total = 0;
        for (int i = 0; i < 5; i++) {
            total += points[i];
        }
        return total;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();
        sc.nextLine(); // consume leftover newline

        Player[] players = new Player[N];

        for (int i = 0; i < N; i++) {
            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();

            int[] pts = new int[5];
            for (int j = 0; j < 5; j++) {
                pts[j] = sc.nextInt();
            }

            players[i] = new Player(id, name, pts);
        }

        Player topScorer = players[0];
        for (int i = 0; i < N; i++) {
            int totalScore = players[i].calculateTotalScore();

```

```
System.out.println("Player ID: " + players[i].getPlayerId());
System.out.println("Player Name: " + players[i].getPlayerName());
System.out.println("Total Score: " + totalScore);
```

```
if (totalScore > topScorer.calculateTotalScore() ||
    (totalScore == topScorer.calculateTotalScore() &&
     players[i].getPlayerId() < topScorer.getPlayerId())) {
```

```
    topScorer = players[i];
```

```
}
```

```
}
```

```
System.out.println("Top Scorer: " + topScorer.getPlayerName() +
    " with " + topScorer.calculateTotalScore() + " points");
```

```
sc.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is working as a developer for CityWater Supply Board, which wants to build a household water billing system.

Each household's water account has:

A Customer ID (integer) A Customer Name (string) Liters Consumed (double)

The water bill is calculated based on these rules:

For the first 500 liters 2 per liter For the next 500 liters (501–1000) 3 per liter
For liters above 1000 5 per liter If the total bill exceeds 3000, a 10% discount is applied on the final bill.

Arjun has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize

customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Liters Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

300

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 600.0

Answer

```
// You are using Java
import java.util.Scanner;
```

```
class Customer {
    private int customerId;
    private String customerName;
    private double litersConsumed;

    // Constructor
    public Customer(int customerId, String customerName, double
    litersConsumed) {
        this.customerId = customerId;
        this.customerName = customerName;
        this.litersConsumed = litersConsumed;
    }

    // Getters
    public int getCustomerId() {
        return customerId;
    }

    public String getCustomerName() {
        return customerName;
    }

    public double getLitersConsumed() {
        return litersConsumed;
    }

    // Setters
    public void setCustomerId(int customerId) {
        this.customerId = customerId;
    }

    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }

    public void setLitersConsumed(double litersConsumed) {
        this.litersConsumed = litersConsumed;
    }

    // Method to calculate final bill
    public double calculateBill() {
```

```

double bill = 0;
double liters = litersConsumed;

if (liters <= 500) {
    bill = liters * 2;
}
else if (liters <= 1000) {
    bill = (500 * 2) + ((liters - 500) * 3);
}
else {
    bill = (500 * 2) + (500 * 3) + ((liters - 1000) * 5);
}

// Apply 10% discount if bill exceeds 3000
if (bill > 3000) {
    bill = bill - (bill * 0.10);
}

return bill;
}
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < N; i++) {

            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();

            double liters = sc.nextDouble();

            Customer customer = new Customer(id, name, liters);

            double finalBill = customer.calculateBill();

```



```
        System.out.println("Customer ID: " + customer.getId());
        System.out.println("Customer Name: " + customer.getName());
        System.out.printf("Final Bill: %.1f\n", finalBill);
    }

    sc.close();
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Anjali is now working as a developer for the City Marathon Association, which wants to build a system to track and find the fastest runner among marathon participants.

Each runner's record has:

Runner ID (integer) Runner Name (string) An array of times (in minutes) taken in 5 marathon events (integers)

The system must calculate:

The average time of each runner (sum of all times / 5). Identify the fastest runner (the one with the lowest average time). If two or more runners have the same average time, the one with the lower Runner ID is considered the fastest runner.

Anjali has been asked to implement this system using:

A class with attributes for runner details. A constructor to initialize runner details. Getter and Setter methods to retrieve and update runner details if required. A method to calculate the average time. Objects of the class to represent runners.

Finally, display each runner's details and announce the Fastest Runner.

Input Format

The first line of input contains an integer N (number of runners).

For each runner:

- The next line contains the Runner ID (integer).
- The following line contains the Runner Name (string).
- The next line contains 5 integers separated by spaces (times in minutes for 5 marathon events).

Output Format

For each runner the output prints the following details:

- Runner ID: <runner_id>
- Runner Name: <runner_name>
- Average Time: <average_time>

Finally, print "Fastest Runner: <runner_name> with <average_time> minutes"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

240 250 245 255 260

Output: Runner ID: 1001

Runner Name: Ravi Kumar

Average Time: 250

Fastest Runner: Ravi Kumar with 250 minutes

Answer

```
// You are using Java
import java.util.Scanner;
```

```
class Runner {
    private int runnerId;
    private String runnerName;
```

```
private int[] times;
```

```
// Constructor
```

```
public Runner(int runnerId, String runnerName, int[] times) {  
    this.runnerId = runnerId;  
    this.runnerName = runnerName;  
    this.times = times;  
}
```

```
// Getters
```

```
public int getRunnerId() {  
    return runnerId;  
}
```

```
public String getRunnerName() {  
    return runnerName;  
}
```

```
public int[] getTimes() {  
    return times;  
}
```

```
// Setters
```

```
public void setRunnerId(int runnerId) {  
    this.runnerId = runnerId;  
}
```

```
public void setRunnerName(String runnerName) {  
    this.runnerName = runnerName;  
}
```

```
public void setTimes(int[] times) {  
    this.times = times;  
}
```

```
// Method to calculate average time
```

```
public int calculateAverageTime() {  
    int sum = 0;  
    for (int i = 0; i < 5; i++) {  
        sum += times[i];  
    }  
    return sum / 5; // integer average (as per sample output)
```

```

    }
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        sc.nextLine(); // consume newline

        Runner[] runners = new Runner[N];

        for (int i = 0; i < N; i++) {
            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();

            int[] times = new int[5];
            for (int j = 0; j < 5; j++) {
                times[j] = sc.nextInt();
            }

            runners[i] = new Runner(id, name, times);
        }

        Runner fastest = runners[0];

        for (int i = 0; i < N; i++) {

            int avgTime = runners[i].calculateAverageTime();

            System.out.println("Runner ID: " + runners[i].getRunnerId());
            System.out.println("Runner Name: " + runners[i].getRunnerName());
            System.out.println("Average Time: " + avgTime);

            if (avgTime < fastest.calculateAverageTime() ||
                (avgTime == fastest.calculateAverageTime() &&
                 runners[i].getRunnerId() < fastest.getRunnerId())) {

                fastest = runners[i];
            }
        }
    }
}

```

```

    }
}

System.out.println("Fastest Runner: " + fastest.getRunnerName() +
    " with " + fastest.calculateAverageTime() + " minutes");

sc.close();
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Meera is working as a developer for CityGas Supply Board, which wants to build a household gas billing system.

Each household's gas account has:

A Customer ID (integer) A Customer Name (string) Units Consumed in cubic meters (double)

The gas bill is calculated based on these rules:

For the first 50 units 4 per unit For the next 100 units (51–150) 6 per unit
 For units above 150 8 per unit If the total bill exceeds 2000, a 15% discount is applied on the final bill.

Meera has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (The final bill must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

30

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 120.0

Answer

// You are using Java

```
import java.util.Scanner;
```

```
class Customer {  
    private int customerId;  
    private String customerName;  
    private double unitsConsumed;
```

```
    // Constructor
```

```
    public Customer(int customerId, String customerName, double  
unitsConsumed) {  
        this.customerId = customerId;
```

```
this.customerName = customerName;
this.unitsConsumed = unitsConsumed;
}

// Getters
public int getCustomerId() {
    return customerId;
}

public String getCustomerName() {
    return customerName;
}

public double getUnitsConsumed() {
    return unitsConsumed;
}

// Setters
public void setCustomerId(int customerId) {
    this.customerId = customerId;
}

public void setCustomerName(String customerName) {
    this.customerName = customerName;
}

public void setUnitsConsumed(double unitsConsumed) {
    this.unitsConsumed = unitsConsumed;
}

// Method to calculate final bill
public double calculateBill() {
    double bill = 0;
    double units = unitsConsumed;

    if (units <= 50) {
        bill = units * 4;
    }
    else if (units <= 150) {
        bill = (50 * 4) + ((units - 50) * 6);
    }
    else {
```

```

        bill = (50 * 4) + (100 * 6) + ((units - 150) * 8);
    }

    // Apply 15% discount if bill exceeds 2000
    if (bill > 2000) {
        bill = bill - (bill * 0.15);
    }

    return bill;
}
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        sc.nextLine(); // consume newline

        for (int i = 0; i < N; i++) {

            int id = sc.nextInt();
            sc.nextLine();

            String name = sc.nextLine();

            double units = sc.nextDouble();

            Customer customer = new Customer(id, name, units);

            double finalBill = customer.calculateBill();

            System.out.println("Customer ID: " + customer.getId());
            System.out.println("Customer Name: " + customer.getName());
            System.out.printf("Final Bill: %.1f\n", finalBill);
        }

        sc.close();
    }
}

```

Status : Correct

Marks : 10/10